

Quiz — 1.2.2 Applications Generation

Name: _____ Date: _ **Mark:** ____ / 25

OCR H446 · Component 01 · 1.2.2

Section A — Multiple choice (8 marks)

1. Which of the following is an example of **utility software**? A. Web browser B. Word processor C. Disk defragmenter D. Computer game Your answer: ☐
 2. A key feature of **open source** software is that: A. The source code is hidden and cannot be modified B. The source code is available and may be modified C. It always requires a paid licence D. It can only be supported by the original vendor Your answer: ☐
 3. A **compiler** differs from an **interpreter** because it: A. Translates and executes one line at a time B. Produces a standalone executable, translating all at once before running C. Requires the source code to be present every time the program runs D. Stops at the first error it finds Your answer: ☐
 4. An **assembler** translates: A. High-level code into machine code all at once B. Assembly language into machine code ($\approx$ one-to-one) C. Machine code into high-level code D. Bytecode into source code Your answer: ☐
 5. In which stage of compilation are **syntax errors** reported? A. Lexical analysis B. Syntax analysis (parsing) C. Code generation D. Code optimisation Your answer: ☐
 6. During **lexical analysis** the source code is: A. Checked against the grammar to build a parse tree B. Converted into tokens with whitespace/comments removed C. Turned into final machine code D. Refined to run faster Your answer: ☐
 7. What is the purpose of a **loader**? A. To combine modules and libraries into one executable B. To copy the executable from storage into RAM ready to run C. To translate one line of source at a time D. To remove dead code from the program Your answer: ☐
 8. A **dynamic** link (e.g. a .dll) differs from a static link because it: A. Copies all library code into the executable, making it larger B. Is linked at run time and can be shared/updated separately C. Removes the need for any libraries D. Can only be used with interpreted languages Your answer: ☐
-

Section B — Short answer (12 marks)

9. State **one** difference between application software and utility software, giving an example of each. [2]

10. The four stages of compilation are listed below in the wrong order. Write them out **in the correct order**. [2]

1. ____ 2. ____

2. ____ 4. ____

11. Explain **two** advantages of using an interpreter rather than a compiler when developing and debugging a program. [2]

12. Explain what a **library** is and give **two** reasons a programmer might use one. [3]

13. Describe the role of a **linker** and explain the difference between **static** and **dynamic** linking. [3]

Section C — Exam-style question (5 marks)

14. A developer has finished writing a program and wants to distribute it to customers who must not be able to see or change the source code, and who should not need any extra software to run it. Explain why a **compiler** is more appropriate than an **interpreter** for this purpose. [5]

Answer key

Section A (8 marks — 1 each) 1. **C** — a defragmenter maintains/optimises the computer itself → utility software. 2. **B** — open source means source code is available and modifiable (usually free, community support). 3. **B** — a compiler translates all at once before running and produces a standalone executable. 4. **B** — an assembler converts assembly mnemonics to machine code, roughly one-to-one. 5. **B** — syntax errors are detected during syntax analysis (parsing) against the grammar. 6. **B** — lexical analysis turns source into tokens, removing whitespace/comments and building a symbol table. 7. **B** — the loader copies the executable from storage into RAM and prepares it to run. 8. **B** — dynamic links are separate files linked at run time, smaller, shareable and updatable.

Section B (12 marks)

9. [2] Application software lets the user perform a real-world task; utility software maintains/optimises the computer itself (1). Example pair, e.g. word processor / browser (application) and antivirus / defrag / backup (utility) (1).

10. **[2]** Correct order: **1. Lexical analysis** → **2. Syntax analysis** → **3. Code generation** → **4. Code optimisation**. (2 if all four correct; 1 if one pair transposed.)
11. **[2]** Any two (1 each): translates line by line so it **stops at the first error**, making errors easier to locate; no need to recompile the whole program after each change (faster edit-test cycle); code is portable/can run on any machine with the interpreter; good for testing partial programs.
12. **[3]** A library is a collection of **pre-written, pre-compiled and tested** subroutines (1). Reasons (1 each, max 2): saves development time / avoids rewriting code; more reliable because the code is already tested; promotes reuse/consistency.
13. **[3]** A linker combines the compiled program with library code and other modules into a single executable, resolving references between them (1). **Static** linking copies the library code into the executable (larger, self-contained) (1); **dynamic** linking keeps the library separate (e.g. .dll) and links it at run time (smaller, shared, updatable) (1).

Section C (5 marks) 14. **[5]** Up to 5 from: a compiler translates the whole program into machine code **before** distribution, producing a standalone **executable** (1); only the .exe is shipped, so customers receive machine code, not source — protecting the **source code** from being seen or modified (1); the executable runs **without** needing the compiler/interpreter or source present (1), so customers need no extra software (1); it also runs **faster** as it is already machine code (1). (An interpreter would require the source plus the interpreter at run time, exposing the code — hence unsuitable.)

Total: 8 + 12 + 5 = 25